

程序设计 II 第 12 周 题目汇总（完整版）

来源：Matrix · 课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设II-周 12-课后 1

类型：编程题

题面

直线与圆的交点数量

以点 Point 类为基类，重新派生类 Line、Circle。

坐标原点为(0,0)，圆的圆心默认为(0,0),只定义半径。

函数 num_point 判断圆和直线的交点数量

直线由 $Ax + By + Cz = 0$ 表示

备注：

根据两点 (x_1, y_1) , (x_2, y_2) 得到 $Ax + By + Cz = 0$ 的方式：

$$A = y_2 - y_1$$

$$B = x_1 - x_2$$

$$C = x_2 y_1 - x_1 y_2$$

类的定义如下，你只需要提交这些类的定义和实现。

```
class Point{
    private:
        double x,y;
    public:
        Point();
        Point(double xv,double yv);
        Point(Point& pt);

        double getx();
        double gety();
};
```

```
class Line: public Point {
    private:
```

```

        double a,b,c;
    public:
        Line(Point& p1,Point& p2); //根据两点坐标转化为直线的一般式，定义 a,b,c

        double geta();
        double getb();
        double getc();
};

class Circle: public Point {
    private:
        double radius;
    public:
        Circle(double r);
        double getRadius();
};

int num_point(Circle& cir, Line& line) // 判断直线和圆的交点数量

```

输入示例 1

4 2 5 7 3

输出示例 1

Point Num is 0

输入示例 2

4 45 16 22 70

输出示例 2

Point Num is 2

提交文件：shape.h

平台提供的初始文件

文件 main.cpp：

```

#include<iostream>
#include"shape.h"
using namespace std;

int main(){
    int x1,x2,y1,y2,r;
    cin >> x1 >> y1 >> x2 >> y2 >> r;
    Point p1(x1,y1),p2(x2,y2);
    Line line(p1,p2);
    Circle cir(r);
    int num = num_point(cir,line);
    cout <<"Point Num is " << num <<endl;
}

```

```
    return 0;
}
```

25 程设II-周 12-课后 2

类型：编程题

题面

消息处理系统

你需要实现一个消息处理系统，模拟服务器处理不同类型的消息（**文本消息**和**紧急消息**）。每条消息有一个处理时间，系统按照消息类型优先处理。模拟器基于多态实现，处理消息并输出处理结果。消息通过固定大小的数组管理（最大 100 条消息）。系统持续运行，直到所有消息处理完成。

任务

1. 实现抽象基类 **Message**，包含纯虚函数 **process()**（处理消息，返回是否完成）、**getId()**（返回消息 ID）和 **isUrgent()**（判断是否为紧急消息）。
2. 派生类 **TextMessage**（文本消息，每帧 **process** 减少 1 单位时间）和 **UrgentMessage**（紧急消息，每帧 **process** 减少 2 单位时间）。
3. 使用 **MessageQueue** 单例管理消息数组。
4. 主循环处理输入的消息序列，每帧处理所有消息，按照 id 顺序优先处理 **UrgentMessage**（减少 2 单位时间），再处理 **TextMessage**。若有完成消息，则输出完成消息的 ID（减少 1 单位时间），直到所有消息处理完成。
5. 如果消息处理时间耗尽（ ≤ 0 ），从数组移除并输出其 ID（标记为已移除）。

输入

- 第一行：整数 **N**（消息数量， $1 \leq N \leq 100$ ）。
- 接下来 **N** 行，每行描述一条消息：消息类型（T 表示文本消息，U 表示紧急消息）、消息 **ID**（ $1 \leq ID \leq 1000$ ）、处理时间（ $1 \leq \text{time} \leq 100$ ）。

输出

- 对于每帧，首先输出“**Frame:**”（帧数从 1 开始递增），后跟换行。
- 如果有消息处理完成，输出完成消息的 ID（按处理顺序，紧急消息先于文本消息），每个 ID 占一行。
- 每帧输出之间用空行分隔。

输入示例：

```
2
T 1 2
U 2 3
```

输出示例：

Frame: 1

Frame: 2

2

1

提交文件：solution.cpp

平台提供的初始文件

文件 messages.h：

```
#ifndef MESSAGES_H
#define MESSAGES_H
#include <iostream>
```

```
class Message {
public:
    virtual bool process() = 0;
    virtual int getId() const = 0;
    virtual bool isUrgent() const = 0;
    virtual ~Message() {}
};
```

```
class TextMessage : public Message {
protected:
    int id, time;
public:
    TextMessage(int id_, int time_);
    bool process() override;
    int getId() const override;
    bool isUrgent() const override;
};
```

```
class UrgentMessage : public Message {
protected:
    int id, time;
public:
    UrgentMessage(int id_, int time_);
    bool process() override;
    int getId() const override;
    bool isUrgent() const override;
};
```

```
class MessageQueue {
private:
    Message* messages[100];
    bool active[100];
    int messageCount;
```

```

static MessageQueue* instance;
MessageQueue() : messageCount(0) {
    for (int i = 0; i < 100; ++i) {
        messages[i] = nullptr;
        active[i] = false;
    }
}
public:
    static MessageQueue& getInstance();
    void addMessage(Message* message);
    bool hasActiveMessages() const;
    void processMessages();
    ~MessageQueue();
};

#endif

```

文件 main.cpp :

```

#include "messages.h"
#include <iostream>

using namespace std;

int main() {
    int N;
    cin >> N;
    MessageQueue& queue = MessageQueue::getInstance();
    for (int i = 0; i < N; ++i) {
        char type;
        int id, time;
        cin >> type >> id >> time;
        if (type == 'T') {
            queue.addMessage(new TextMessage(id, time));
        } else {
            queue.addMessage(new UrgentMessage(id, time));
        }
    }
    int frame = 1;
    while (queue.hasActiveMessages()) {
        cout << "Frame: " << frame << endl;
        queue.processMessages();
        if (queue.hasActiveMessages()) cout << endl;
        ++frame;
    }
    return 0;
}

```

25 程设II-周 12-课后 3

类型：编程题

题面

宝可梦管理系统

题目描述

设计一个宝可梦管理系统，要求实现以下类层次结构：

完善基类 宝可梦 *Pokemon*：

- 保护成员：string name、int level、int hp
- 公有成员：
 - *Pokemon*(const string& n, int lvl, int health)：构造函数，初始化
 - void display()：const 虚函数。
 - double calculatePower()：const 纯虚函数。
 - void train()

display

const 虚函数。打印格式如下：

名称: {name}
等级: {level}
生命值: {hp}

train

训练宝可梦。宝可梦的 level 加 1；hp 加 5。并输出{name}训练后等级提升至{level}

定义 *Pokemon* 的派生类 妙蛙种子 *Bulbasaur*：

- 保护成员：double vineLength
- 公有成员：void display() (const)、double calculatePower() (const)

display (const) 打印格式在*Pokemon* 基础上末尾添加：藤鞭长度: {vineLength}

calculatePower (const) *Bulbasaur* 的计算方式为 $level * hp$

定义 *Bulbasaur* 的派生类 妙蛙草 *Ivysaur*：

- 私有成员：double photosynthesisAbility
- 公有成员：void display() (const)、double calculatePower() (const)、bool hasBloom() (const)

hasBloom : photosynthesisAbility $\geq$ 1.0 则为 true , 否则 false

display (const) 末尾添加 : 花苞状态: {真->"已开放" ; 假->"未开放"} 和 光合作用能力: {photosynthesisAbility}

*calculatePower (const) : hasBloom 真:level*hp*2 + photosynthesisAbility*100 ;
假:level*hp + photosynthesisAbility*100*

完善宝可梦管理类 **PokemonCenter**

完善 void evolvePokemon(int idx) : 若是 Bulbasaur 且 level $\geq$ 10 进化为 Ivysaur (输出 {name}进化为 Ivysaur! , 进化后 hp+100、vineLength $\times$ 1.5、level 不变、photosynthesisAbility=0) , 否则输出{name}等级不够, 无法进化! ; 若已是 Ivysaur 输出 {name}已经是 Ivysaur,无法再进化!。

提示

1. 用 using 改变访问控制 ; 2. 思考虚函数/默认实现/重载复用 ; 3. 自行设计构造函数 ; 4. 可用 dynamic_cast 检测类型 ; 5. 构造最小测试 ; 6. 在实现文件最上方加 **#pragma once**。

输入示例 1

```
0
Bulbasaur1 1 100 0.7
0
Bulbasaur2 3 200 1
2
```

输出示例

所有宝可梦信息:
名称: Bulbasaur1
等级: 1
生命值: 100
藤鞭长度: 0.7 米

名称: Bulbasaur2
等级: 3
生命值: 200
藤鞭长度: 1 米

输入示例 2

```
0
Bulbasaur1 10 100 0.7
4
0
0
Bulbasaur2 3 200 1
4
1
1
Ivysaur1 3 200 1 1.2
```

4
2

输出示例 2

Bulbasaur1 进化为 Ivysaur!

Bulbasaur2 等级不够，无法进化!

Ivysaur1 已经是 Ivysaur,无法再进化!

提交文件： Pokemon.cpp 、 PokemonCenter.cpp 、 Bulbasaur.hpp 、 Ivysaur.hpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include "PokemonCenter.h"
#include "Pokemon.h"
#include "Bulbasaur.hpp"
#include "Ivysaur.hpp"
```

```
using namespace std;
```

```
int main() {
    PokemonCenter center;

    int cmd;
    string name;
    int hp;
    int level;
    double length;
    double ability;
    int idx;
    while(cin>>cmd) {
        switch(cmd) {
            case 0: {
                cin>>name>>level>>hp>>length;
                center.addPokemon(new Bulbasaur(name, level, hp, length));
                break;
            }
            case 1: {
                cin>>name>>level>>hp>>length>>ability;
                center.addPokemon(new Ivysaur(name, level, hp, length, ability));
                break;
            }
            case 2: {
                center.displayAll();
            }
        }
    }
}
```



```

        break;
    }
    case 3: {
        center.calculatePower();
        break;
    }
    case 4: {
        cin>>idx;
        center.evolvePokemon(idx);
        break;
    }
    case 5: {
        cin>>idx;
        center.pokemons[idx]->train();
        break;
    }
    default:
        break;
}
}
return 0;
}

```

文件 Pokemon.h :

```

#pragma once
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

```

```
using namespace std;
```

```
// 基类Pokemon
```

```
class Pokemon {
protected:
```

```
    string name;
    int level;
    int hp;

```

```
public:
```

```
    Pokemon(const string& n) : name(n), level(1), hp(0) {}

```

```
    Pokemon(const string& n, int level, int hp) : name(n), level(level), hp(hp) {}

```

```
// 虚析构造函数确保正确释放派生类对象
```

```
virtual ~Pokemon() {}

```

```
// 显示信息的虚函数
```

```
virtual void display() const;
```

```

// 计算战斗力的虚函数
virtual double calculatePower() const = 0;

void train();

int getLevel() const {return this->level;}

int getHP() const {return this->hp;}
};

```

文件 PokemonCenter.h :

```

#pragma once
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include "Pokemon.h"
#include "Bulbasaur.hpp"
#include "Ivysaur.hpp"
using namespace std;

```

```

// 宝可梦中心管理类
class PokemonCenter {
public:
    Pokemon* pokemons[1000];
    int num_pokemons = 0;
    ~PokemonCenter() {
        for (int i = 0; i < this->num_pokemons; i++) {
            delete this->pokemons[i];
        }
    }

    void addPokemon(Pokemon* p) {
        this->pokemons[num_pokemons++] = p;
    }

    void displayAll() const {
        cout << "所有宝可梦信息:" << endl;
        for (int i = 0; i < this->num_pokemons; i++) {
            this->pokemons[i]->display();
            cout << endl;
        }
    }

    void calculatePower() const {
        cout << "计算战斗力:" << endl;
        for (int i = 0; i < this->num_pokemons; i++) {

```

```

        auto p = this->pokemons[i];
        cout << dynamic_cast<Bulbasaur*>(p)->name << "("
            << (dynamic_cast<Ivysaur*>(p) ? "Ivysaur" :
                (dynamic_cast<Bulbasaur*>(p) ? "Bulbasaur" : "Pokemon"))
            << ")战斗力: " << p->calculatePower() << endl;
    }
    cout << endl;
}

void evolvePokemon(int idx);
};

```

25 程设II-周 12-课后 4

类型：编程题

题面

宝可梦对战模拟器

宝可梦具备自身的面板属性（属性、生命值 hp、攻击力、防御力、速度）以及自身的特殊技能，设计一个包含初代御三家的宝可梦对战系统。

宝可梦面板属性（1 级）

宝可梦

小火龙 Charmander

杰尼龟 Squirtle

妙蛙种子 Bulbasaur

每升一级增加：Charmander +20/+10/+30/+15；Squirtle +25/+30/+25/+20；Bulbasaur +30/+20/+15/+30（生命/速度/攻击/防御）。

属性克制

火克草、草克水、水克火。克制方攻击力 +50%。

技能伤害

- Charmander 喷射火焰：伤害 = $50 * \lfloor \text{自身攻击力} / \text{对手防御力} \rfloor$
- Squirtle 水之波动：伤害 = $30 * \lfloor \text{自身攻击力} / \text{对手防御力} \rfloor + 20$
- Bulbasaur 飞叶快刀：伤害 = $20 * \lfloor \text{自身攻击力} / \text{对手防御力} \rfloor$ 且自身 hp+20

对战过程

速度高者先攻，交替攻击直到一方 $hp \leq 0$ ；50 回合未分胜负则平局。

基类 **Pokemon** : 保护成员 **name, hp, type, speed, attack, defense, level** ; 公有
纯虚 **void upgrade()** 、**void attackTo(Pokemon* other)** 。派生
Charmander/Squirtle/Bulbasaur 实现构造、升级、攻击。

输入：第一行 宝可梦 1 名称、等级；第二行 宝可梦 2 名称、等级。

输入示例：

Charmander 5
Squirtle 4

输出示例：

Charmander[5]:(280)
s:340 a:420 d:160
Squirtle[4]:(275)
s:290 a:325 d:210
Charmander(280) VS Squirtle(275)
Charmander(280) => Squirtle(175)
Charmander(170) <= Squirtle(175)
Charmander(170) => Squirtle(75)
Charmander(60) <= Squirtle(75)
Charmander(60) => Squirtle(-25)
Winner : Charmander

提交文件：pokemon.cpp

平台提供的初始文件

文件 main.cpp：

```
#include "pokemon.hpp"
```

```
Pokemon* createPokemon(string pok, int lev) {  
    Pokemon *poke = nullptr;  
    if (pok == "Charmander") poke = new Charmander;  
    else if (pok == "Squirtle") poke = new Squirtle;  
    else poke = new Bulbasaur;  
    while (--lev) {  
        poke->upgrade();  
    }  
    return poke;  
}  
  
void VS(Pokemon* a, Pokemon* b) {  
    printf("%s(%d) VS %s(%d)\n", a->name.c_str(), a->hp, b->name.c_str(), b->hp);  
    int turn = 0, cnt = 1;  
    if (a->speed < b->speed) turn = 1;  
    while(a->hp > 0 && b->hp > 0 && cnt <= 100) {  
        if (turn == 0) {  
            a->attackTo(b);  
            printf("%s(%d) => %s(%d)\n", a->name.c_str(), a->hp, b->name.c_str(), b->hp);  
            turn = 1;  
        }  
        else {  
            b->attackTo(a);  
            printf("%s(%d) => %s(%d)\n", b->name.c_str(), b->hp, a->name.c_str(), a->hp);  
            turn = 0;  
        }  
        cnt++;  
    }  
}
```

```

    }
    else {
        b->attackTo(a);
        printf("%s(%d) <= %s(%d)\n", a->name.c_str(), a->hp, b->name.c_str(), b-
>hp);
    }
    turn = turn ^ 1;
    cnt += 1;
}
if (b->hp <= 0) printf("Winner : %s\n", a->name.c_str());
else if (a->hp <= 0) printf("Winner : %s\n", b->name.c_str());
else printf("Draw\n");
}

```

```

int main() {
    Pokemon *poke1, *poke2;
    string p1, p2;
    int level1, level2;
    cin >> p1 >> level1 >> p2 >> level2;
    poke1 = createPokemon(p1, level1);
    poke2 = createPokemon(p2, level2);
    poke1->printInfo();
    poke2->printInfo();
    VS(poke1, poke2);
    delete(poke1);
    delete(poke2);
    return 0;
}

```

文件 pokemon.hpp :

```

#ifndef POKEMON_HPP
#define POKEMON_HPP

#include <iostream>
#include <cmath>
using namespace std;

enum TYPE {fire = 0, water, grass};

class Pokemon {
private:
    string name;
    int hp;
    TYPE type;
    int speed;
    int attack;
    int defense;
    int level = 1;
public:

```

```

Pokemon(string name, int hp, TYPE type, int speed, int attack, int defense) {
    this->name = name;
    this->hp = hp;
    this->type = type;
    this->speed = speed;
    this->attack = attack;
    this->defense = defense;
}
virtual ~Pokemon() {}
virtual void upgrade() = 0;
virtual void attackTo(Pokemon* other) = 0;

void printInfo() {
    printf("%s[%d]:(%d)\n", name.c_str(), level, hp);
    printf("s:%d a:%d d:%d\n", speed, attack, defense);
}
friend class Charmander;
friend class Squirtle;
friend class Bulbasaur;
friend void VS(Pokemon* a, Pokemon* b);
};

class Charmander : public Pokemon {
public:
    Charmander();
    void upgrade();
    void attackTo(Pokemon* other);
};

class Squirtle : public Pokemon {
public:
    Squirtle();
    void upgrade();
    void attackTo(Pokemon* other);
};

class Bulbasaur : public Pokemon {
public:
    Bulbasaur();
    void upgrade();
    void attackTo(Pokemon* other);
};

#endif

```

25 程设II-周 12-课堂 1

类型：编程题

题面

圆柱的表面积和体积

以点 Circle 和 Rectangle 类为基类，重新派生类 Cylinder。

并根据 Circle 和 Rectangle 的相关属性计算圆柱的体积和表面积(上底 + 下底 + 侧面)。(PI 取 3.14)

```
class Circle{
private:
    double radius;
public:
    Circle(double r);
    Circle(Circle& cir);

    double circle_area(); // 计算圆的面积
    double getr(); //返回半径
};

class Rectangle: {
private:
    double width;
    double height;
public:
    Rectangle(double w, double h);
    Rectangle(Rectangle& rec);

    double rectangle_area(); //计算矩形面积
    double getw(); // 返回 width 参数
    double geth(); // 返回 height 参数
};

class Cylinder: public Circle, public Rectangle{
public:
    Cylinder(double radius,double w, double h);
    Cylinder(Circle& cir,Rectangle& rec);
    double area();
    double volume();
    double display();//按照格式输出圆柱的表面积和体积
};
```

输入示例 1

1 2 3 4 5 6

输出示例 1

Cylinder: Area is 12.28 Volume is 9.42
Cylinder: Area is 130.48 Volume is 301.44

输入示例 2

3 12 5 45 1 23

输出示例 2

Cylinder: Area is 116.52 Volume is 141.3

Cylinder: Area is 12740 Volume is 146246

提交文件：Cylinder.h

平台提供的初始文件

文件 main.cpp：

```
#include "Cylinder.h"
#include <iostream>
using namespace std;
```

```
int main() {
    int r1,w1,h1,r2,w2,h2;
    cin >> r1 >> w1 >> h1;
    cin >> r2 >> w2 >> h2;
    Cylinder cyl1(r1,w1,h1);
    Circle cir(r2);
    Rectangle rec(w2,h2);
    cyl1.cylinder_show();
    Cylinder cyl2(cir,rec);
    cyl2.cylinder_show();
    return 0;
}
```

25 程设II-周 12-课堂 2

类型：编程题

题面

形状边界反弹模拟器

在一个 2D 模拟系统中，形状在一个宽度为 W 的画布上沿水平方向移动，遇到边界 ($x = 0$ 或 $x = W - 1$) 会反弹。系统使用抽象类 **Shape** 定义纯虚函数 **move()**，派生类 **Circle** 和 **Rectangle** 实现不同速度的移动和反弹行为。实现 **ShapeManager** 类，使用固定大小数组管理形状，通过多态更新位置并处理反弹。

任务

1. 抽象类 **Shape**：保护成员 x （整数）、**direction**（1 右/-1 左）；纯虚 **virtual void move(int W) = 0**；虚析构。

2. 派生类 **Circle**：构造接受初始 x 和 $direction$ ；**move(int W)** 速度 1，越界 ($x < 0$ 或 $x \geq W$) 反弹并反转 $direction$ 。
3. 派生类 **Rectangle**：构造同上；**move(int W)** 速度 2，越界反弹反转。
4. 单例类 **ShapeManager**：私有 **Shape* shapes[100]**、**int count**；公有 **getInstance()**、**addShape**、**moveShapes(int W)**、**drawShapes()**、析构删除所有形状。

输入：第一行 N 、 W ；第二行 T (tick 次数)；接下来 N 行 $type(C/R)$ 、 x 、 dir 。

输出：按添加顺序输出每个形状最终 x 位置。

输入示例

```
2 10
3
C 0 1
R 8 -1
```

输出示例

```
3
2
```

提交文件：shapes.cpp

平台提供的初始文件

文件 main.cpp：

```
#include "shapes.h"
#include <iostream>
using namespace std;

int main() {
    int N, W, T;
    cin >> N >> W >> T;
    ShapeManager& manager = ShapeManager::getInstance();

    for (int i = 0; i < N; ++i) {
        char type;
        int x, dir;
        cin >> type >> x >> dir;
        if (type == 'C') {
            manager.addShape(new Circle(x, dir));
        } else if (type == 'R') {
            manager.addShape(new Rectangle(x, dir));
        }
    }

    for (int i = 0; i < T; ++i) {
        manager.moveShapes(W);
    }
}
```

```
    manager.drawShapes();  
    return 0;  
}
```

文件 shapes.h :

```
#ifndef SHAPES_H  
#define SHAPES_H  
  
class Shape {  
protected:  
    int x;  
    int direction;  
public:  
    virtual void move(int W) = 0;  
    virtual ~Shape() {}  
    int getX() const { return x; }  
};  
  
class Circle : public Shape {  
public:  
    Circle(int x, int dir);  
    void move(int W) override;  
};  
  
class Rectangle : public Shape {  
public:  
    Rectangle(int x, int dir);  
    void move(int W) override;  
};  
  
class ShapeManager {  
private:  
    Shape* shapes[100];  
    int count;  
    static ShapeManager* instance;  
    ShapeManager() : count(0) {}  
public:  
    static ShapeManager& getInstance();  
    void addShape(Shape* shape);  
    void moveShapes(int W);  
    void drawShapes();  
    ~ShapeManager();  
};  
  
#endif
```

25 程设II-周 12-理论题

类型：选择题（20 题）

第 1 题

在 C++ 中，多态意味着？

- A. 只具有单一形态的类
- B. 编译时决定行为的类
- C. 允许对象表现出多种行为形态的特性
- D. 仅具备静态行为的类

答案：C

第 2 题

以下哪个概念允许你在 C++ 中重用已编写的代码？

- A. 继承
- B. 多态
- C. 抽象
- D. 封装

答案：A

第 3 题

在 C++ 中，虚函数的主要目的是什么？

- A. 实现运行时多态
- B. 减少代码量
- C. 保证数据的安全性
- D. 提高代码的执行效率

答案：A

第 4 题

虚函数是如何实现动态绑定（或称为迟绑定）的？

- A. 通过在编译时确定函数地址
- B. 仅在静态类型为基类时使用
- C. 通过在每个类中复制函数代码
- D. 通过在对象中添加指向虚函数表的指针

答案：D

第 5 题

下列关于虚函数的描述，哪一项是错误的

- A. 虚函数可以在构造函数中被调用
- B. 虚函数可以是静态成员函数
- C. 虚函数可以在纯虚类中声明
- D. 虚函数可以被覆盖（override）在派生类中

答案：B

第 6 题

多态的含义是？

- A. 同一个接口，多种实现方式
- B. 不同的接口，相同的实现
- C. 一种特定的编程技巧，提高代码复用
- D. 一种数据结构，用于存储多种类型的数据

答案：A

第 7 题

在 C++ 中，函数重载是基于什么原则来区分不同版本的函数？

- A. 函数名相同且参数列表不同
- B. 函数名不同但返回类型相同
- C. 函数名相同且返回类型不同
- D. 函数名和参数列表都不同

答案：A

第 8 题

以下哪些是实现运行时多态的方式？

- A. 函数重载
- B. 虚函数
- C. 重写（Override）
- D. 模板函数

答案：B,C

第 9 题

关于动态类型，以下哪个说法是正确的？

- A. 动态类型是在编译时确定的
- B. 动态类型可以通过向下转型来改变

- C. 动态类型决定了实际调用哪个函数版本
- D. 动态类型与静态类型总是相同的

答案：C

第 10 题

函数重载与函数覆盖（Override）的主要区别是什么？

- A. 重载仅限于非成员函数，覆盖则是成员函数特有的现象
- B. 重载函数必须有相同的返回类型，覆盖则不必
- C. 覆盖函数必须有相同的函数签名，重载则不必
- D. 重载发生在不同文件中，覆盖必须在同一文件内进行

答案：C

第 11 题

以下哪种情况会导致编译错误？

- A. 派生类重写基类的虚函数
- B. 基类定义纯虚函数但未实现
- C. 实例化抽象类
- D. 基类指针指向派生类对象

答案：C

第 12 题

在 C++ 中，override 关键字的作用是什么？

- A. 声明一个函数为虚函数
- B. 确保派生类函数正确重写基类虚函数
- C. 阻止派生类重写虚函数
- D. 将函数标记为纯虚函数

答案：B

第 13 题

以下代码中，哪个选项正确描述运行时行为？

```
class Base {
public:
    virtual void func() { std::cout << "Base"; }
};
class Derived : public Base {
public:
    void func() override { std::cout << "Derived"; }
};
```

```
Derived d;  
Base b = d;  
b.func();
```

- A. 输出“Derived”
- B. 输出“Base”
- C. 编译错误
- D. 运行时崩溃

答案：B

第 14 题

纯虚函数的定义中不能包含：

- A. 参数
- B. 返回值
- C. 函数体
- D. 修饰符 virtual

答案：C

第 15 题

以下哪个函数声明表明“该函数为最终覆盖，禁止后续派生类再次覆盖”？

- A. virtual void f() final;
- B. virtual void f();
- C. virtual void f() override;
- D. virtual void f() const;

答案：A

第 16 题

下列代码输出结果为？

```
class Base {  
public:  
    virtual void print() { std::cout << "Base\n"; }  
};  
class Derived : public Base {  
public:  
    void print() { std::cout << "Derived\n"; }  
};  
void call(Base& b) {  
    b.print();  
}  
int main() {  
    Derived d;
```

```
    call(d);  
}
```

- A. 输出 “Derived”
- B. 输出 “Base”
- C. 编译错误
- D. 输出未定义

答案：A

第 17 题

C++中实现运行时多态必须满足哪两个条件？

- A. 成员函数为 virtual + 通过对象访问
- B. 函数名相同 + 类名不同
- C. 返回值类型相同 + 参数个数不同
- D. 成员函数为 virtual + 通过指针或引用访问

答案：D

第 18 题

以下哪个概念允许你在 C++中重用已编写的代码？

- A. 继承
- B. 多态
- C. 抽象
- D. 封装

答案：A

第 19 题

下列关于动态绑定的说法正确的是？

- A. 使用基类指针或引用调用虚函数时才会发生动态绑定
- B. 非虚函数在运行时进行动态绑定
- C. 虚函数必须被 override 才会动态绑定
- D. 所有函数默认都是动态绑定的

答案：A

第 20 题

关于多态，说法不正确的是（）

- A. C++语言的多态性分为编译时的多态性和运行时的多态性
- B. 编译时的多态性可通过函数重载实现
- C. 运行时的多态性可通过模板和虚函数实现

- D. 实现运行时多态性的机制称为动态绑定

答案：c
